

Informe de Implementacion de Tests segun README

Segunda Evaluacion PSP - Sistema de Monitoreo Ambiental

Proyecto Java EE, Pipe-Filter, JUnit 4, Mockito, ArchUnit y Vert.x

23 de junio de 2026

Índice

1. Proposito del informe	2
2. Criterio general para evitar tests redundantes	2
3. Estructura recomendada de tests	2
4. Dependencias necesarias en Maven	3
5. Tests de filtros: validacion, normalizacion y extremos	3
5.1. Test 1: validacion correcta de datos	3
5.2. Test 2: errores de formato o entradas invalidas	4
5.3. Test 3: normalizacion de unidades	5
5.4. Test 4: deteccion de valores extremos	6
6. Tests del servicio Pipe-Filter con Mockito	7
6.1. Objetivo	7
6.2. Test 5: pipeline exitoso y persistencia invocada	7
6.3. Test 6: si un filtro falla, el dato no se guarda	8
7. Tests de repositorio e insercion en base de datos	9
7.1. Test 7: insercion exitosa en base de datos	9
7.2. Test 8: RawDataRepository.findAll retorna datos	10
8. Tests arquitecturales con ArchUnit	10
8.1. Regla R1: ningun filtro salvo el final accede a base de datos	10
8.2. Regla R2: toda clase filtro implementa RawDataFilter	11
9. Tests del flujo Vert.x Pub/Sub	12
9.1. Responsabilidad exigida por el README	12
9.2. Test 9: flujo completo publica en validated.data	12
9.3. Test 10: dato invalido no llega a validated.data	13
9.4. Test 11: los verticles de filtro son estandar y no acceden a BD	14
10. Lista final de tests recomendados	14
11. Orden recomendado de implementacion	15
12. Conclusion	15

1. Proposito del informe

Este informe detalla como implementar todos los tests exigidos o recomendados por el README del proyecto. El README solicita tres grupos principales de verificacion:

1. **Verificacion arquitectural automatica:** comprobar reglas R1 y R2 sobre filtros y acceso a base de datos.
2. **Pruebas unitarias con JUnit 4:** al menos cinco casos, incluyendo Mockito sin anotaciones.
3. **Pruebas del flujo reactivo con Vert.x:** validar que los verticles se comuniquen por EventBus y no por llamadas directas.

El proyecto base contiene las entidades `RawData` y `CleanData`, los repositorios `RawDataRepository` y `CleanDataRepository`, el servicio `RawDataProcessingService`, la interfaz `RawDataFilter` y los verticles iniciales `ReaderBDVerticle`, `ProducerBDVerticle` y `FileStorageVerticle`. Sin embargo, para que los tests sean completos, se asume que se implementan filtros concretos para validacion, normalizacion y deteccion de extremos.

2. Criterio general para evitar tests redundantes

La regla metodologica debe ser:

Una regla se prueba donde vive.

Por ejemplo, si la validacion de tipo permitido se encuentra en `ValidatorFilter`, entonces se prueba en `ValidatorFilterTest`. No debe repetirse la misma validacion en `RawDataProcessingServiceTest`, salvo que el servicio agregue una regla adicional, como descartar el dato y no persistirlo cuando un filtro falla.

Lugar de la regla	Responsabilidad	Test correcto
<code>ValidatorFilter</code>	Rechazar datos nulos, incompletos o con tipo invalido	<code>ValidatorFilterTest</code>
<code>UnitNormalizerFilter</code>	Convertir unidades a estandar	<code>UnitNormalizerFilterTest</code>
<code>ExtremeValueFilter</code>	Rechazar valores fuera de rango	<code>ExtremeValueFilterTest</code>
<code>RawDataProcessingService</code>	Coordinar filtros y persistencia	<code>RawDataProcessingServiceTest</code>
<code>CleanDataRepository</code>	Insertar en base de datos	<code>CleanDataRepositoryTest</code>
Arquitectura	Cumplir R1 y R2	<code>ArchitectureTest</code>
Verticles	Publicar y consumir por EventBus	<code>VertxPipelineTest</code>

3. Estructura recomendada de tests

Se recomienda crear la siguiente estructura bajo `src/test/java`:

```
src/test/java/  
  adhoc/  
    filters/  
      ValidatorFilterTest.java  
      UnitNormalizerFilterTest.java  
      ExtremeValueFilterTest.java  
    repository/  
      CleanDataRepositoryTest.java  
      RawDataRepositoryTest.java  
    service/  
      RawDataProcessingServiceTest.java  
    architecture/  
      ArchitectureTest.java  
  vertx/
```

Listing 1: Estructura recomendada de pruebas

Si se mantiene todo dentro del paquete `adhoc`, los tests tambien pueden quedar directamente bajo `src/test/java/adhoc`; sin embargo, separar por subpaquetes vuelve mas clara la evaluacion.

4. Dependencias necesarias en Maven

El `pom.xml` ya contiene dependencias compatibles con lo pedido: JUnit 4, Mockito y ArchUnit. Para las pruebas Vert.x tambien aparece `vertx-junit5`; no obstante, como el README pide JUnit 4 para las pruebas unitarias, conviene que los tests principales sean JUnit 4. Para Vert.x se puede usar JUnit 4 con `CountDownLatch`, evitando mezclar innecesariamente estilos.

```
<dependency>
  <groupId>junit</groupId>
  <artifactId>junit</artifactId>
  <version>4.13.2</version>
  <scope>test</scope>
</dependency>

<dependency>
  <groupId>org.mockito</groupId>
  <artifactId>mockito-core</artifactId>
  <version>4.11.0</version>
  <scope>test</scope>
</dependency>

<dependency>
  <groupId>com.tngtech.archunit</groupId>
  <artifactId>archunit-junit4</artifactId>
  <version>1.4.1</version>
  <scope>test</scope>
</dependency>
```

Listing 2: Dependencias clave ya presentes en el proyecto

5. Tests de filtros: validacion, normalizacion y extremos

5.1. Test 1: validacion correcta de datos

Este test cubre el punto del README: *Validacion correcta de datos*. La responsabilidad pertenece al filtro `ValidatorFilter`. Un dato valido debe pasar sin excepcion y conservar sus atributos.

```
package adhoc.filters;

import adhoc.RawData;
import org.junit.Test;

import java.time.LocalDateTime;

import static org.junit.Assert.assertEquals;

public class ValidatorFilterTest {

    @Test
    public void datoValidoDeTemperaturaDebePasar() throws Exception {
        ValidatorFilter filter = new ValidatorFilter();
        RawData input = new RawData(
            "temperature",
            LocalDateTime.of(2025, 6, 21, 10, 0),
```

```

        25.0,
        "C"
    );

    RawData result = filter.apply(input);

    assertEquals("temperature", result.getType());
    assertEquals(25.0, result.getValue(), 0.0001);
    assertEquals("C", result.getUnit());
}

@Test
public void datoValidoDeMaterialParticuladoDebePasar() throws Exception {
    ValidatorFilter filter = new ValidatorFilter();
    RawData input = new RawData(
        "mp",
        LocalDateTime.of(2025, 6, 21, 10, 0),
        55.0,
        "ug/m3"
    );

    RawData result = filter.apply(input);

    assertEquals("mp", result.getType());
    assertEquals(55.0, result.getValue(), 0.0001);
    assertEquals("ug/m3", result.getUnit());
}
}

```

Listing 3: ValidatorFilterTest - dato valido

5.2. Test 2: errores de formato o entradas invalidas

Este test cubre el punto del README: *Errores de formato o entradas invalidas*. Deben rechazarse lecturas incompletas o con tipo diferente a `temperature` o `mp`.

```

@Test(expected = IllegalArgumentException.class)
public void debeRechazarTipoNulo() throws Exception {
    ValidatorFilter filter = new ValidatorFilter();
    RawData input = new RawData(null, LocalDateTime.now(), 20.0, "C");

    filter.apply(input);
}

@Test(expected = IllegalArgumentException.class)
public void debeRechazarTimestampNulo() throws Exception {
    ValidatorFilter filter = new ValidatorFilter();
    RawData input = new RawData("temperature", null, 20.0, "C");

    filter.apply(input);
}

@Test(expected = IllegalArgumentException.class)
public void debeRechazarTipoInvalido() throws Exception {
    ValidatorFilter filter = new ValidatorFilter();
    RawData input = new RawData("humidity", LocalDateTime.now(), 60.0, "%");

    filter.apply(input);
}

```

Listing 4: ValidatorFilterTest - entradas invalidas

5.3. Test 3: normalizacion de unidades

Este test cubre la conversion exigida por el README:

$$C = \frac{F - 32}{1.8}$$
$$\mu g/m^3 = mg/m^3 \times 1000$$

Para evitar caracteres problematicos en codigo fuente, se recomienda representar microgramos como ug/m3.

```
package adhoc.filters;

import adhoc.RawData;
import org.junit.Test;

import java.time.LocalDateTime;

import static org.junit.Assert.assertEquals;

public class UnitNormalizerFilterTest {

    @Test
    public void debeConvertirFahrenheitACelsius() throws Exception {
        UnitNormalizerFilter filter = new UnitNormalizerFilter();
        RawData input = new RawData("temperature", LocalDateTime.now(), 68.0, "F")
            ;

        RawData result = filter.apply(input);

        assertEquals("temperature", result.getType());
        assertEquals(20.0, result.getValue(), 0.0001);
        assertEquals("C", result.getUnit());
    }

    @Test
    public void debeMantenerTemperaturaEnCelsius() throws Exception {
        UnitNormalizerFilter filter = new UnitNormalizerFilter();
        RawData input = new RawData("temperature", LocalDateTime.now(), 24.0, "C")
            ;

        RawData result = filter.apply(input);

        assertEquals(24.0, result.getValue(), 0.0001);
        assertEquals("C", result.getUnit());
    }

    @Test
    public void debeConvertirMgM3AUgM3() throws Exception {
        UnitNormalizerFilter filter = new UnitNormalizerFilter();
        RawData input = new RawData("mp", LocalDateTime.now(), 0.082, "mg/m3");

        RawData result = filter.apply(input);

        assertEquals("mp", result.getType());
        assertEquals(82.0, result.getValue(), 0.0001);
        assertEquals("ug/m3", result.getUnit());
    }
}
```

Listing 5: UnitNormalizerFilterTest

5.4. Test 4: deteccion de valores extremos

Este test cubre el punto del README: *Filtros que detecten valores extremos*. Los rangos son:

Variable	Minimo	Maximo
Temperatura	-50 C	70 C
Material particulado	0 ug/m3	1000 ug/m3

Conviene probar valores internos, bordes validos y valores fuera de rango.

```
package adhoc.filters;

import adhoc.RawData;
import org.junit.Test;

import java.time.LocalDateTime;

import static org.junit.Assert.assertEquals;

public class ExtremeValueFilterTest {

    @Test
    public void debeAceptarTemperaturaEnBordes() throws Exception {
        ExtremeValueFilter filter = new ExtremeValueFilter();

        RawData min = new RawData("temperature", LocalDateTime.now(), -50.0, "C");
        RawData max = new RawData("temperature", LocalDateTime.now(), 70.0, "C");

        assertEquals(-50.0, filter.apply(min).getValue(), 0.0001);
        assertEquals(70.0, filter.apply(max).getValue(), 0.0001);
    }

    @Test(expected = IllegalArgumentException.class)
    public void debeRechazarTemperaturaMenorAlMinimo() throws Exception {
        ExtremeValueFilter filter = new ExtremeValueFilter();
        filter.apply(new RawData("temperature", LocalDateTime.now(), -51.0, "C"));
    }

    @Test(expected = IllegalArgumentException.class)
    public void debeRechazarTemperaturaMayorAlMaximo() throws Exception {
        ExtremeValueFilter filter = new ExtremeValueFilter();
        filter.apply(new RawData("temperature", LocalDateTime.now(), 70.1, "C"));
    }

    @Test
    public void debeAceptarMaterialParticuladoEnBordes() throws Exception {
        ExtremeValueFilter filter = new ExtremeValueFilter();

        RawData min = new RawData("mp", LocalDateTime.now(), 0.0, "ug/m3");
        RawData max = new RawData("mp", LocalDateTime.now(), 1000.0, "ug/m3");

        assertEquals(0.0, filter.apply(min).getValue(), 0.0001);
        assertEquals(1000.0, filter.apply(max).getValue(), 0.0001);
    }

    @Test(expected = IllegalArgumentException.class)
    public void debeRechazarMaterialParticuladoNegativo() throws Exception {
        ExtremeValueFilter filter = new ExtremeValueFilter();
        filter.apply(new RawData("mp", LocalDateTime.now(), -0.1, "ug/m3"));
    }

    @Test(expected = IllegalArgumentException.class)
    public void debeRechazarMaterialParticuladoMayorA1000() throws Exception {
        ExtremeValueFilter filter = new ExtremeValueFilter();
    }
}
```

```

        filter.apply(new RawData("mp", LocalDateTime.now(), 1000.1, "ug/m3"));
    }
}

```

Listing 6: ExtremeValueFilterTest

6. Tests del servicio Pipe-Filter con Mockito

6.1. Objetivo

El README exige que al menos una prueba use Mockito sin anotaciones, es decir, mediante `Mockito.mock(...)`. El mejor lugar para usarlo es `RawDataProcessingServiceTest`, porque el servicio coordina repositorios y filtros.

Este test no debe repetir la logica de cada filtro. Debe probar que:

- lee datos desde `RawDataRepository`;
- aplica la cadena de filtros;
- si todo pasa, transforma `RawData` en `CleanData`;
- llama a `CleanDataRepository.save(...)`;
- si un filtro falla, no guarda el dato.

6.2. Test 5: pipeline exitoso y persistencia invocada

```

package adhoc.service;

import adhoc.CleanData;
import adhoc.CleanDataRepository;
import adhoc.RawData;
import adhoc.RawDataFilter;
import adhoc.RawDataProcessingService;
import adhoc.RawDataRepository;
import org.junit.Test;
import org.mockito.ArgumentCaptor;

import java.time.LocalDateTime;
import java.util.List;

import static org.junit.Assert.assertEquals;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.mock;
import static org.mockito.Mockito.verify;
import static org.mockito.Mockito.when;

public class RawDataProcessingServiceTest {

    @Test
    public void processAllDebeGuardarDatoCuandoTodosLosFiltrosPasan() throws
        Exception {
        RawDataRepository rawRepo = mock(RawDataRepository.class);
        CleanDataRepository cleanRepo = mock(CleanDataRepository.class);

        RawData raw = new RawData(
            "temperature",
            LocalDateTime.of(2025, 6, 21, 10, 0),
            68.0,
            "F"
        );

        RawData normalized = new RawData(
            "temperature",

```

```

        raw.getTimestamp(),
        20.0,
        "C"
    );

    RawDataFilter filter = mock(RawDataFilter.class);

    when(rawRepo.findAll()).thenReturn(List.of(raw));
    when(filter.apply(raw)).thenReturn(normalized);

    RawDataProcessingService service = new RawDataProcessingService(
        rawRepo,
        cleanRepo,
        List.of(filter)
    );

    service.processAll();

    ArgumentCaptor<CleanData> captor = ArgumentCaptor.forClass(CleanData.class);
    verify(cleanRepo).save(captor.capture());

    CleanData saved = captor.getValue();
    assertEquals("temperature", saved.getType());
    assertEquals(20.0, saved.getValue(), 0.0001);
}
}

```

Listing 7: RawDataProcessingServiceTest con Mockito sin anotaciones

6.3. Test 6: si un filtro falla, el dato no se guarda

Este caso no duplica la validacion de ValidatorFilter. Prueba una regla propia del servicio: si cualquier filtro lanza excepcion, el dato se descarta y no se persiste.

```

@Test
public void processAllNoDebeGuardarCuandoUnFiltroFalla() throws Exception {
    RawDataRepository rawRepo = mock(RawDataRepository.class);
    CleanDataRepository cleanRepo = mock(CleanDataRepository.class);
    RawDataFilter filter = mock(RawDataFilter.class);

    RawData raw = new RawData("humidity", LocalDateTime.now(), 50.0, "%");

    when(rawRepo.findAll()).thenReturn(List.of(raw));
    when(filter.apply(raw)).thenThrow(new IllegalArgumentException("tipo invalido"));

    RawDataProcessingService service = new RawDataProcessingService(
        rawRepo,
        cleanRepo,
        List.of(filter)
    );

    service.processAll();

    org.mockito.Mockito.verify(cleanRepo, org.mockito.Mockito.never())
        .save(any(CleanData.class));
}

```

Listing 8: RawDataProcessingServiceTest - fallo en filtro

7. Tests de repositorio e insercion en base de datos

7.1. Test 7: insercion exitosa en base de datos

El README exige probar *Insercion exitosa en la base de datos*. Este test debe ser de integracion ligera, no unitario puro, porque usa JPA y SQLite.

Para no contaminar la base `environment.db`, se recomienda crear una base temporal para la prueba.

```
package adhoc.repository;

import adhoc.CleanData;
import adhoc.CleanDataRepository;
import jakarta.persistence.EntityManager;
import jakarta.persistence.EntityManagerFactory;
import jakarta.persistence.Persistence;
import org.junit.After;
import org.junit.Before;
import org.junit.Test;

import java.nio.file.Files;
import java.nio.file.Path;
import java.time.LocalDateTime;
import java.util.HashMap;
import java.util.Map;

import static org.junit.Assert.assertEquals;

public class CleanDataRepositoryTest {

    private EntityManagerFactory emf;
    private Path dbFile;

    @Before
    public void setUp() throws Exception {
        dbFile = Files.createTempFile("clean-data-test", ".db");

        Map<String, Object> props = new HashMap<>();
        props.put("jakarta.persistence.jdbc.url", "jdbc:sqlite:" + dbFile.
            toAbsolutePath());
        props.put("jakarta.persistence.schema-generation.database.action", "drop-
            and-create");
        props.put("jakarta.persistence.sql-load-script-source", "");

        emf = Persistence.createEntityManagerFactory("environment", props);
    }

    @After
    public void tearDown() throws Exception {
        if (emf != null && emf.isOpen()) {
            emf.close();
        }
        Files.deleteIfExists(dbFile);
    }

    @Test
    public void saveDebeInsertarCleanDataEnLaBase() {
        CleanDataRepository repository = new CleanDataRepository(emf);
        CleanData data = new CleanData(
            "temperature",
            LocalDateTime.of(2025, 6, 21, 12, 0),
            24.5
        );
    }
}
```

```

        repository.save(data);

        EntityManager em = emf.createEntityManager();
        Long count = em.createQuery(
            "select count(c) from CleanData c",
            Long.class
        ).getSingleResult();
        em.close();

        assertEquals(Long.valueOf(1L), count);
    }
}

```

Listing 9: CleanDataRepositoryTest - insercion exitosa

7.2. Test 8: RawDataRepository.findAll retorna datos

La lista de cotejo asigna puntaje al metodo `RawDataRepository.findAll()`. Por tanto, conviene testearlo explicitamente una vez implementado.

```

@Test
public void findAllDebeRetornarLecturasCrudas() {
    RawDataRepository repository = new RawDataRepository(emf);

    List<RawData> result = repository.findAll();

    org.junit.Assert.assertNotNull(result);
    org.junit.Assert.assertFalse(result.isEmpty());
}

```

Listing 10: RawDataRepositoryTest - findAll

Este test requiere que la base de prueba tenga datos cargados. Puede usarse `data.sql` o insertar manualmente una fila en el `@Before`.

8. Tests arquitecturales con ArchUnit

8.1. Regla R1: ningun filtro salvo el final accede a base de datos

El README establece:

R1: Ningun filtro, excepto el ultimo filtro aplicado, debe acceder directamente a las clases que gestionan los datos en la base de datos.

Hay dos formas validas de modelar esto:

1. **Persistencia fuera de los filtros:** `RawDataProcessingService` guarda al final. En ese caso, ningun filtro debe depender de repositorios ni JPA.
2. **Persistencia como ultimo filtro:** se crea un `StorageFilter` o `DatabaseStorageFilter`. En ese caso, solo ese filtro puede depender de `CleanDataRepository`.

La primera alternativa calza con el proyecto base actual, donde `RawDataProcessingService` llama a `cleanRepo.save(clean)`. La regla ArchUnit quedaria asi:

```

package adhoc.architecture;

import adhoc.RawDataFilter;
import com.tngtech.archunit.core.domain.JavaClasses;
import com.tngtech.archunit.core.importer.ClassFileImporter;
import org.junit.Test;

```

```
import static com.tngtech.archunit.lang.syntax.ArchRuleDefinition.noClasses;

public class ArchitectureTest {

    @Test
    public void filtrosNoDebenAccederAREpositoriosNiJpa() {
        JavaClasses classes = new ClassFileImporter().importPackages("adhoc");

        noClasses()
            .that().implement(RawDataFilter.class)
            .should().dependOnClassesThat()
            .resideInAnyPackage(
                "..repository..",
                "jakarta.persistence.."
            )
            .because("los filtros deben ser etapas puras del pipeline, sin
                acceso directo a BD")
            .check(classes);
    }
}
```

Listing 11: ArchitectureTest - filtros sin acceso a BD

Si se decide implementar un ultimo filtro de almacenamiento, la regla se ajusta con una excepcion nominal:

```
noClasses()
    .that().implement(RawDataFilter.class)
    .and().haveSimpleNameNotEndingWith("StorageFilter")
    .should().dependOnClassesThat()
    .resideInAnyPackage("..repository..", "jakarta.persistence..");
```

Listing 12: R1 con excepcion para DatabaseStorageFilter

8.2. Regla R2: toda clase filtro implementa RawDataFilter

El README establece:

R2: Toda clase de filtro debe implementar la interfaz RawDataFilter.

Para que esta regla sea precisa, conviene ubicar los filtros en un paquete identificable, por ejemplo `adhoc.filters`. Luego se define que toda clase dentro de ese paquete, cuyo nombre termine en `Filter`, debe implementar `RawDataFilter`.

```
import static com.tngtech.archunit.lang.syntax.ArchRuleDefinition.classes;

@Test
public void todaClaseFilterDebeImplementarRawDataFilter() {
    JavaClasses classes = new ClassFileImporter().importPackages("adhoc");

    classes()
        .that().resideInAPackage("..filters..")
        .and().haveSimpleNameEndingWith("Filter")
        .should().implement(RawDataFilter.class)
        .because("el README exige que toda clase de filtro implemente
            RawDataFilter")
        .check(classes);
}
```

Listing 13: ArchitectureTest - todo filtro implementa RawDataFilter

9. Tests del flujo Vert.x Pub/Sub

9.1. Responsabilidad exigida por el README

El README pide una migración a Vert.x donde cada filtro sea un `Verticle` estándar, sin `WorkerVerticle`, conectado exclusivamente por `EventBus`. El flujo requerido es:

```
raw.data.incoming
  -> ValidatorFilterVerticle
filter.validated
  -> UnitNormalizerFilterVerticle
filter.normalized
  -> ExtremeValueFilterVerticle
validated.data
  -> FileStorageVerticle
```

Listing 14: Flujo reactivo requerido

9.2. Test 9: flujo completo publica en validated.data

Este test comprueba que una lectura válida entra por `raw.data.incoming` y sale por `validated.data`. Se usa `CountDownLatch` para esperar asincrónicamente el mensaje.

```
package vertx;

import io.vertx.core.Vertx;
import io.vertx.core.json.JsonObject;
import org.junit.After;
import org.junit.Before;
import org.junit.Test;

import java.time.LocalDateTime;
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.atomic.AtomicReference;

import static org.junit.Assert.assertEquals;
import static org.junit.Assert.assertTrue;

public class VertxPipelineTest {

    private Vertx vertx;

    @Before
    public void setUp() {
        vertx = Vertx.vertx();
    }

    @After
    public void tearDown() throws Exception {
        CountDownLatch latch = new CountDownLatch(1);
        vertx.close(ar -> latch.countDown());
        latch.await(2, TimeUnit.SECONDS);
    }

    @Test
    public void flujoCompletoDebePublicarDatoValidado() throws Exception {
        CountDownLatch deploymentLatch = new CountDownLatch(3);

        vertx.deployVerticle(new ValidatorFilterVerticle(), ar -> deploymentLatch.countDown());
        vertx.deployVerticle(new UnitNormalizerFilterVerticle(), ar -> deploymentLatch.countDown());
```

```

        vertx.deployVerticle(new ExtremeValueFilterVerticle(), ar ->
            deploymentLatch.countDown());

        assertTrue(deploymentLatch.await(2, TimeUnit.SECONDS));

        AtomicReference<JsonObject> output = new AtomicReference<>();
        CountDownLatch messageLatch = new CountDownLatch(1);

        vertx.eventBus().consumer("validated.data", message -> {
            output.set((JsonObject) message.body());
            messageLatch.countDown();
        });

        JsonObject input = new JsonObject()
            .put("type", "temperature")
            .put("timestamp", LocalDateTime.of(2025, 6, 21, 10, 0).toString())
            .put("value", 68.0)
            .put("unit", "F");

        vertx.eventBus().publish("raw.data.incoming", input);

        assertTrue(messageLatch.await(2, TimeUnit.SECONDS));
        assertEquals("temperature", output.get().getString("type"));
        assertEquals(20.0, output.get().getDouble("value"), 0.0001);
        assertEquals("C", output.get().getString("unit"));
    }
}

```

Listing 15: VertxPipelineTest - flujo completo con JUnit 4

9.3. Test 10: dato invalido no llega a validated.data

Este test verifica que el pipeline reactivo descarta datos invalidos. Si se publica una lectura con `type = humidity`, no debe aparecer nada en `validated.data`.

```

@Test
public void datoInvalidoNoDebeLlegarAValidatedData() throws Exception {
    CountDownLatch deploymentLatch = new CountDownLatch(3);

    vertx.deployVerticle(new ValidatorFilterVerticle(), ar -> deploymentLatch.
        countDown());
    vertx.deployVerticle(new UnitNormalizerFilterVerticle(), ar -> deploymentLatch
        .countDown());
    vertx.deployVerticle(new ExtremeValueFilterVerticle(), ar -> deploymentLatch.
        countDown());

    assertTrue(deploymentLatch.await(2, TimeUnit.SECONDS));

    CountDownLatch messageLatch = new CountDownLatch(1);

    vertx.eventBus().consumer("validated.data", message -> messageLatch.countDown
        ());

    JsonObject input = new JsonObject()
        .put("type", "humidity")
        .put("timestamp", LocalDateTime.now().toString())
        .put("value", 80.0)
        .put("unit", "%");

    vertx.eventBus().publish("raw.data.incoming", input);

    boolean received = messageLatch.await(500, TimeUnit.MILLISECONDS);
    org.junit.Assert.assertFalse(received);
}

```

9.4. Test 11: los verticles de filtro son estandar y no acceden a BD

El README dice que los filtros Vert.x deben ser verticles estandar, sin `WorkerVerticle`, y que la version reactiva es *solo Verticles, sin BD*. Esto puede verificarse con ArchUnit.

```
@Test
public void verticlesDeFiltroNoDebenDependerDeJpaNiRepositorios() {
    JavaClasses classes = new ClassFileImporter().importPackages("vertx");

    noClasses()
        .that().resideInAPackage("vertx..")
        .and().haveSimpleNameContaining("FilterVerticle")
        .should().dependOnClassesThat()
        .resideInAnyPackage("jakarta.persistence..", "adhoc..repository..")
        .because("la migracion Vert.x solicitada para filtros no debe acceder a BD")
        .check(classes);
}
```

Listing 17: ArchitectureTest - verticles de filtro sin persistencia

Adicionalmente, la condicion de *sin WorkerVerticle* se valida revisando que los filtros extiendan `AbstractVerticle` y no sean desplegados con `DeploymentOptions.setWorker(true)`. La forma mas simple es no usar `DeploymentOptions` para los filtros:

```
vertx.deployVerticle(new ValidatorFilterVerticle())
    .compose(id -> vertx.deployVerticle(new UnitNormalizerFilterVerticle()))
    .compose(id -> vertx.deployVerticle(new ExtremeValueFilterVerticle()));
```

Listing 18: Despliegue correcto de filtros estandar

10. Lista final de tests recomendados

La bateria recomendada queda compuesta por trece verificaciones. Aunque el README exige un minimo de cinco pruebas unitarias, esta distribucion cubre unidad, integracion ligera, arquitectura y flujo reactivo.

1. `datoValidoDeTemperaturaDebePasar`: valida que el filtro acepte una lectura bien formada.
2. `debeRechazarTipoNulo` / `debeRechazarTipoInvalido`: comprueba errores de formato o entradas invalidas.
3. `debeConvertirFahrenheitACelsius`: verifica la normalizacion de temperatura a grados Celsius.
4. `debeConvertirMgM3AUgM3`: verifica la normalizacion de material particulado a ug/m3.
5. `debeRechazarTemperaturaFueraDeRango`: detecta temperaturas extremas.
6. `debeRechazarMaterialParticuladoFueraDeRango`: detecta valores extremos de material particulado.
7. `processAllDebeGuardarDatoCuandoTodosLosFiltrosPasan`: usa Mockito sin anotaciones y verifica el guardado final.
8. `processAllNoDebeGuardarCuandoUnFiltroFalla`: valida que el servicio descarte lecturas rechazadas.
9. `saveDebeInsertarCleanDataEnLaBase`: comprueba insercion real con JPA y SQLite temporal.
10. `filtrosNoDebenAccederAREpositoriosNiJpa`: verifica la regla arquitectural R1.
11. `todaClaseFilterDebeImplementarRawDataFilter`: verifica la regla arquitectural R2.

12. `flujoCompletoDebePublicarDatoValidado`: prueba que el pipeline Vert.x encadene los filtros por `EventBus`.
13. `datoInvalidoNoDebeLlegarAValidatedData`: comprueba que el pipeline reactivo descarte invalidos.

11. Orden recomendado de implementacion

1. Implementar `RawDataRepository.findAll()` con JPA y probarlo.
2. Crear `ValidatorFilter`, `UnitNormalizerFilter` y `ExtremeValueFilter` implementando `RawDataFilter`.
3. Escribir tests unitarios puros de cada filtro.
4. Escribir `RawDataProcessingServiceTest` con Mockito sin anotaciones.
5. Escribir `CleanDataRepositoryTest` con base SQLite temporal.
6. Escribir `ArchitectureTest` con las reglas R1 y R2.
7. Implementar los tres verticles de filtro: `ValidatorFilterVerticle`, `UnitNormalizerFilterVerticle` y `ExtremeValueFilterVerticle`.
8. Escribir `VertxPipelineTest` para validar el flujo completo por `EventBus`.

12. Conclusion

La implementacion de tests no debe limitarse a cumplir la cantidad minima. Debe demostrar que el sistema respeta las responsabilidades arquitecturales del README: filtros desacoplados, validaciones localizadas, persistencia aislada y pipeline reactivo basado exclusivamente en publicacion y suscripcion.

La bateria recomendada separa correctamente los niveles de prueba:

- los filtros se prueban como unidades puras;
- los repositorios se prueban contra SQLite temporal;
- el servicio se prueba con Mockito para verificar coordinacion;
- ArchUnit valida las reglas R1 y R2;
- Vert.x se prueba observando mensajes reales en el `EventBus`.

Con esta organizacion se evita la redundancia tipica de probar la misma regla en tres capas distintas, y se obtiene una solucion mas defendible frente a la pauta de evaluacion.